서 울 대 학 교
SEOUL NATIONAL UNIVERSITY

Systems Software & Architecture Laboratory
Dept. of Computer Science and Engineering

4190.307: Operating Systems
Midterm Exam. (Spring 2024)

12:30 – 13:45, April 25, 2024. Instructor: Jin-Soo Kim

Student ID: _____

Name: _____

Codename: _____

Q1 (40)		Q4 (60)	
Q2 (45)		Q5 (30)	
Q3 (75)		Q6 (20)	
		Total (270)	

I hereby certify that I have neither given nor received aid in answering the questions on this exam. _____
(Signature)

1. Fill in the blank with a single English word (including hyphenated ones). (4 points each)

(1) Because virtualization allows many programs to run, and many programs to concurrently access their own instructions and data, and many programs to access devices, the OS is sometimes known as a(an) ( ) manager.

(2) To execute a system call, a program must execute a special ( ) instruction. This instruction simultaneously jumps into the kernel and raises the privilege level to kernel mode.

(3) In C programs, the ( ) is used for explicitly requested dynamically-allocated data; programs request such space by calling malloc() and free it explicitly by calling free().

(4) The ( ) time of a job is defined as the time at which the job completes minus the time at which the job arrived in the system.

(5) Avoiding ( ) constants is a good idea whenever possible. Unfortunately, it is often difficult. One could try to make the system learn a good value, but that too is not straightforward.

(6) Protection thus enables us to deliver the property of isolation among processes; such process should be running in its own isolated cocoon, safe from the ravages of other faulty or even ( ) processes.

(7) One solution to the external fragmentation problem would be to ( ) physical memory by rearranging the existing segments.

(8) An even more extreme space savings in the world of page tables is found with ( ) page tables. Here, instead of having many page tables, we keep a single page table that has an entry for each physical page of the system.

(9) A contrast is found in older systems that used memory ( ), which required programmers to manually move pieces of code or data in and out of memory as they were needed.

(10) Deciding which page (or pages) to ( ) is encapsulated within the replacement policy of the OS.

2. Consider a virtual memory system that uses hardware-based dynamic relocation using two registers: a base register and a bounds register. Answer the following questions. Incorrect answers will be penalized. (15 points each)

(1) List all the required "mechanisms" for ***hardware*** to implement virtual memory in this manner.

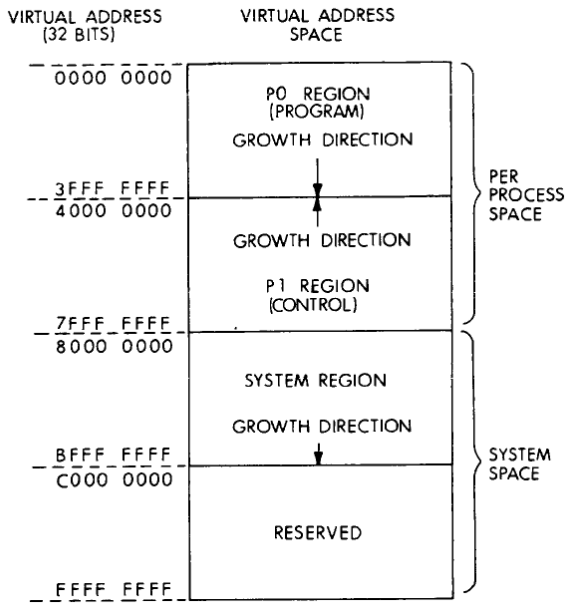
(2) List all the required "mechanisms" for the ***kernel*** to implement virtual memory in this manner.

(3) List all the required "policies" for the ***kernel*** to implement virtual memory in this manner.

3. VAX-11/780 was a 32-bit superminicomputer developed by Digital Equipment Corporation (DEC) in 1977. VAX virtual memory consists of a 32-bit address space that is divided into three regions: two process-space regions (P0 and P1), and the system-space region (S0) used by the kernel. All processes use the same address translation map for system space, so system space is shared among all processes. These regions are selected by the top two bits of the virtual address. Each region is in turn partitioned into 512-byte pages.

A page table is a contiguous array of 32-bit page table entries. The page table for system space, called the System Page Table (SPT), is always located in ***physical*** address space. Therefore, the base address of the SPT is a physical address and is located in the System Base Register (SBR).

The page tables for the P0 and P1 regions have the same structure as SPT, except that they reside at the ***virtual*** address space in the S0 region. Specifically, the P0BR or P1BR register, which points to the start of the P0 or P1 page table, contains a virtual address within the system region. Consequently, translating a P0 or P1 address requires the translation of the S0 page that contains the corresponding P0 or P1 page table entry.



(1) In VAX11/780, the page table is composed of a single level in each region. What is the maximum size of the P0 page table? (10 points)

(2) If the page table is structured as a multi-level page table and the size of each level's page table is adjusted to fit the page size, how many levels of page table are required for the P0 region? (10 points)

(3) In VAX11/780, what are the benefits of locating P0 and P1 page tables in the ***virtual*** address space in the S0 region? (10 points)

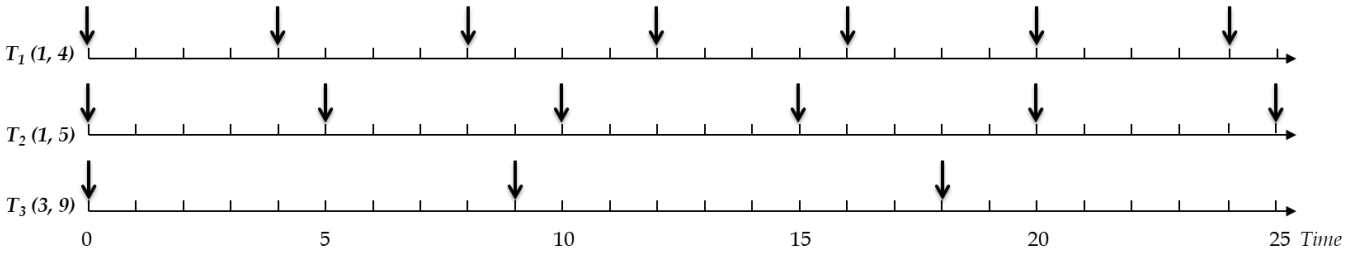
(3) Assume there is no TLB and that swapping is disabled. Describe the step-by-step process of translating an S0 address into a physical address. Also, show how many memory accesses are required just for the translation of the S0 address. (15 points)

(4) Assume there is no TLB and that swapping is disabled. Describe the step-by-step process of translating an P0 address into a physical address. Also, show how many memory accesses are required just for the translation of the P0 address. (20 points)

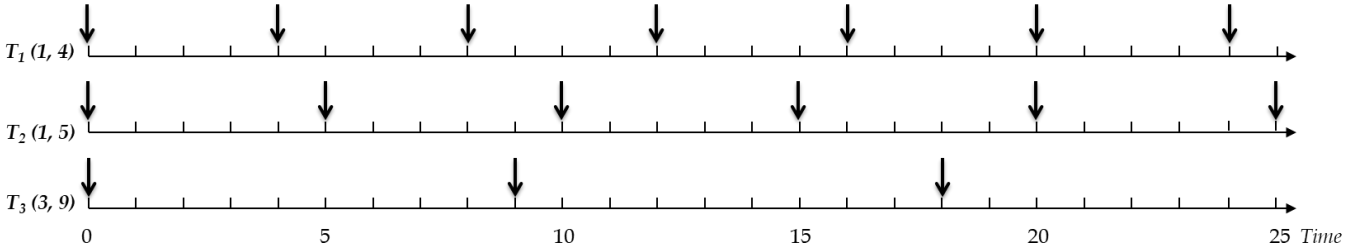
(5) Now assume that the system is equipped with a hardware-managed TLB. When the CPU accesses data from a valid address in the P0 region using a load instruction, what is the maximum number of TLB misses that can occur? Explain Why. Do not count the TLB misses that occur while fetching the load instruction itself. (10 points)

4. A real-time task T is represented as $T = (c, p)$, where c denotes the maximum execution time (in ticks) and p indicates the period (in ticks). Consider the following set of real-time tasks: { $T_1 = (1, 4)$, $T_2 = (1, 5)$, $T_3 = (3, 9)$ }. Assume that T_1 is released (i.e., becomes runnable) periodically at ticks 0, 4, 8, 12, 16, 20, and 24, T_2 at 0, 5, 10, 15, 20, and 25, and T_3 at 0, 9, and 18. Also, assume that each task runs for the maximum execution time once it is scheduled. For the following questions, mark the time slots during which each task is executed. (20 points each)

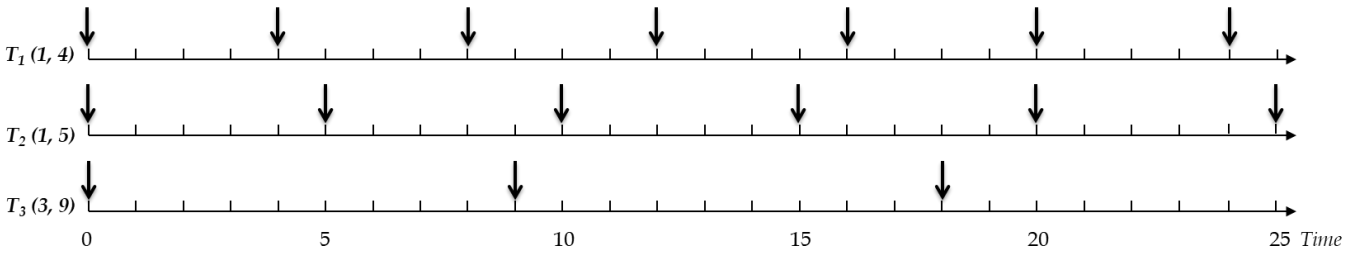
(1) Under the non-preemptive priority scheduling where the priority is set to $T_2 > T_1 > T_3$, how the tasks are scheduled?



(2) Under the preemptive priority scheduling where the priority is set to $T_2 > T_1 > T_3$, how the tasks are scheduled?



(3) Under the EDF scheduling in PA#3, show when each task is scheduled.



5. When a user mode process invokes a system call with the `ecall` instruction in xv6, the kernel should be able to return to the instruction next to the `ecall` instruction in user mode. Answer the following questions. (10 points each)

(1) Where can the xv6 kernel find the information related to the user-mode return address immediately after an `ecall` is invoked?

(2) Where does the xv6 kernel store that information until the completion of the system call?

(3) When returning to the user mode, the value obtained from 5. (1) should be changed? Explain why or why not.

6. What is the trampoline page in xv6? What does it contain? How is it implemented and why do we need it? (20 points)